SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SAVEETHA SCHOOL OF ENGINEERING

ITA06 – MACHINE LEARNING

ASSIGNMENT 2 – CO2

NEURAL NETWORK REPRESENTATION, PERCEPTRON LEARNING, BACKPROPAGATION, DEEP LEARNING AND OPTIMIZATION

Submitted by

T. M. Rishika

192412200

B.E. – Electronics and Communication Engineering

DECLARATION

I, T. M. Rishika (192412200), declare that this report titled “Neural Network Representation, Perceptron Learning, Backpropagation, Deep Learning and Optimization” has been prepared for ITA06 – Machine Learning, Assignment 2 – CO2.

The report has been written in my own words using the assignment brief as the governing specification. Mathematical derivations, code, synthetic experiments, tables and interpretations are presented for academic evaluation. Experimental results are identified with their dataset split, random seed and training configuration wherever applicable.

I understand that academic integrity requires accurate reporting. Synthetic data are not represented as real industrial measurements, and experimental results are not claimed to establish performance in an actual manufacturing plant.

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty members of Saveetha School of Engineering for providing the opportunity to study neural-network learning methods through this assignment. The problem contexts in the assignment helped connect mathematical learning rules with practical classification and monitoring tasks.

I acknowledge the developers and maintainers of Python, NumPy, scikit-learn, PyTorch, Matplotlib and the associated documentation used to implement and evaluate the experiments. I also acknowledge the authors of the textbooks and research resources listed in the references.

Finally, I thank everyone who supported the completion of this academic work.

ABSTRACT

This report studies neural-network design and learning behaviour through a sequence of classification problems. A manufacturing inspection problem is represented using three numerical measurements—surface roughness, dimensional deviation and vibration level—and a binary acceptable/defective label. Two candidate architectures are formulated: a direct logistic output model and a model with one nonlinear hidden layer. Their parameter counts, activation choices and forward computations are derived.

The warehouse dispatch example is used to distinguish a linearly separable AND rule from the XOR review rule. A perceptron learns the first but cannot represent the second using a single linear decision boundary. A two-input, two-hidden-neuron, one-output sigmoid multilayer perceptron is then derived and updated manually using binary cross-entropy gradients. Numerical finite differences are used to check an analytical gradient.

A reproducible two-moons dataset containing 1,000 synthetic observations is used for architecture comparison, regularization, hyperparameter search and redesign. Fixed train, validation and test partitions contain 600, 200 and 200 observations respectively. The report compares one-hidden-layer and three-hidden-layer networks over three seeds, evaluates L2 regularization, dropout and early stopping on a 100-example training subset, and allocates exactly twelve candidate runs to optimization and hyperparameter selection.

The final design challenge diagnoses generalization behaviour and tests a compact regularized network with early stopping. Results are interpreted as evidence from a controlled synthetic experiment rather than a guarantee of manufacturing performance. The report emphasizes parameter accounting, reproducibility, separation of validation from test data and critical interpretation of both successful and unsuccessful learning outcomes.

TABLE OF CONTENTS

- Declaration
- Acknowledgement
- Abstract
- List of Tables and Figures
- Experimental Protocol and Dataset
- Q1 – Neural Network Representation for Manufacturing Inspection
- Q2 – Perceptron Learning and Its Limitations
- Q3 – Backpropagation for a Multilayer Perceptron
- Q4 – Shallow and Deep Network Comparison
- Q5 – Overfitting and Regularization
- Q6 – Hyperparameter and Optimization Strategy
- Q7 – Day-4 Design Challenge
- Overall Conclusion
- References
- Appendix A – Complete Python/NumPy Implementations
- Appendix B – Experimental Configuration and Results
- Appendix C – Dataset Data Dictionary and Sample Records

Note: Page numbers are generated by the word processor when the document is opened or exported. The contents list is organized by assignment question and supporting appendices.

LIST OF TABLES AND FIGURES

- Table 1.1 Manufacturing feature representation
- Table 1.2 Architecture parameter counts
- Table 1.3 Worked forward-pass values
- Table 2.1 Warehouse truth tables
- Table 2.2 Perceptron epoch record
- Table 2.3 XOR learning-rate comparison
- Table 3.1 Forward-pass numerical values
- Table 3.2 Backpropagation gradients
- Table 3.3 Finite-difference check
- Table 4.1 Dataset partition sizes
- Table 4.2 Three-seed architecture comparison
- Table 5.1 Regularization comparison
- Table 6.1 Twelve-run optimization search
- Table 7.1 Baseline and redesign comparison
- Figure 1.1 Direct logistic network
- Figure 1.2 One-hidden-layer network
- Figure 2.1 AND and XOR geometry
- Figure 4.1 Two-moons dataset
- Figure 4.2 Decision boundaries
- Figure 5.1 Validation loss comparison
- Figure 7.1 Redesign workflow

1. EXPERIMENTAL PROTOCOL AND DATASET

1.1 Scope and problem framing

The assignment uses two complementary types of problems. Q1 and Q2 begin with clearly specified binary decision rules. Q3 studies how a hidden layer can learn a nonlinear rule. Q4–Q7 use a synthetic nonlinear classification dataset to compare architecture, regularization and optimization. This progression separates representational questions from training and generalization questions.

For Q1, the manufacturing variables are treated as continuous measurements. For Q2 and Q3, the warehouse indicators are binary. For Q4–Q7, the two-moons dataset is generated synthetically with a fixed random state. The latter is suitable for demonstrating nonlinear boundaries, but it is not a substitute for collecting real manufacturing data.

1.2 Dataset generation

The synthetic dataset is generated with the make_moons procedure using 1,000 observations, two numerical features, noise level 0.22 and random state 42. The two crescent-shaped classes overlap slightly, creating a nonlinear decision boundary. The dataset is split with stratification into 60% training, 20% validation and 20% test data. The resulting counts are 600, 200 and 200.

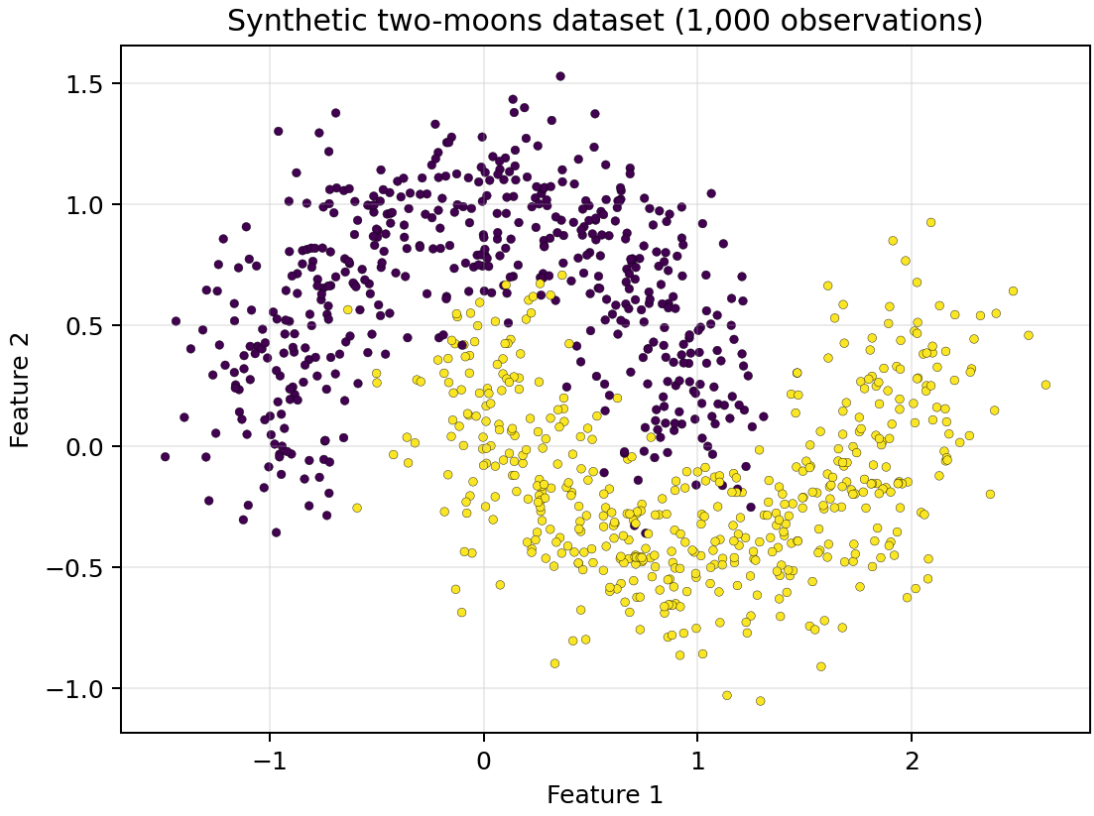


Figure 1.1. Synthetic two-moons dataset generated with 1,000 observations.

1.3 Preprocessing and leakage prevention

Standardization is performed using the training partition only. For each feature, the training mean is subtracted and the result is divided by the training standard deviation. The same fitted transformation is applied to validation and test observations. This prevents information from the validation or test distributions from influencing the fitted preprocessing parameters.

The validation set is used for model selection, early stopping and hyperparameter decisions. The test set is held back until a configuration is selected. A test result is therefore an estimate of final generalization under the synthetic data distribution, not a tuning signal.

Partition	Count	Role
Training	600	Fit model parameters and preprocessing
Validation	200	Select architectures and training configurations
Test	200	One final evaluation after selection

1.4 Reproducibility controls

- Dataset generation seed: 42.
- Q4 comparison seeds: 11, 22 and 33.
- Q5 limited training subset seed: 2026; subset size: 100.
- Q6 candidate runs use distinct, recorded initialization seeds.
- All Q4 architectures use the same fixed partitions and broadly consistent training settings.
- The reported execution times depend on the CPU and software environment and should not be treated as hardware-independent constants.

The CSV companion file contains the original two features, binary label and partition assignment for all 1,000 observations. It allows the data split to be reproduced without reconstructing the random split manually.

2. Q1 – DESIGN A NEURAL NETWORK REPRESENTATION FOR MANUFACTURING QUALITY INSPECTION

2.1 Problem definition

The manufacturing unit records three measurements for every component: surface roughness, dimensional deviation and vibration level. The target is binary: acceptable is encoded as 0 and defective as 1. The model estimates the probability that an inspected component belongs to the defective class.

Let x_1 represent surface roughness, x_2 represent dimensional deviation and x_3 represent vibration level. The target y is 0 or 1. The input vector is $x = [x_1, x_2, x_3]^T$. The numerical units must be retained in the dataset documentation; for example, roughness may be measured in micrometres, dimensional deviation in millimetres and vibration in an agreed acceleration or velocity unit.

Feature	Symbol	Numerical representation	Preprocessing reason
Surface roughness	x_1	Real-valued measurement	Scale may differ from other variables
Dimensional deviation	x_2	Signed real-valued deviation	Preserve positive and negative tolerance deviations
Vibration level	x_3	Non-negative real-valued measurement	Avoid domination solely due to numeric scale
Inspection label	y	0 = acceptable; 1 = defective	Binary target for classification

2.2 Preprocessing

First, measurement units and sensor ranges must be checked. Missing readings should be investigated rather than silently replaced. If a small number of readings are missing and the operational setting permits it, median imputation can be fitted on training data. Obvious sensor faults should be flagged separately because a corrupted measurement is not equivalent to a valid extreme value.

Second, standardization is appropriate because the three inputs may have different units and numerical ranges. For feature j, the standardized value is $z_j = (x_j - \mu_j)/\sigma_j$, where μ_j and σ_j are estimated from the training set. This improves numerical conditioning and makes gradient-based optimization less sensitive to feature scale.

Third, the label must be encoded consistently. The positive class is defective (1). This convention must remain fixed in confusion matrices, recall calculations and threshold decisions.

2.3 Architecture A: no hidden layer

The first candidate is a single logistic output neuron connected directly to the three inputs. It is a linear classifier in the input feature space followed by a sigmoid probability mapping. The network has three input values and one output neuron; there is no hidden layer.

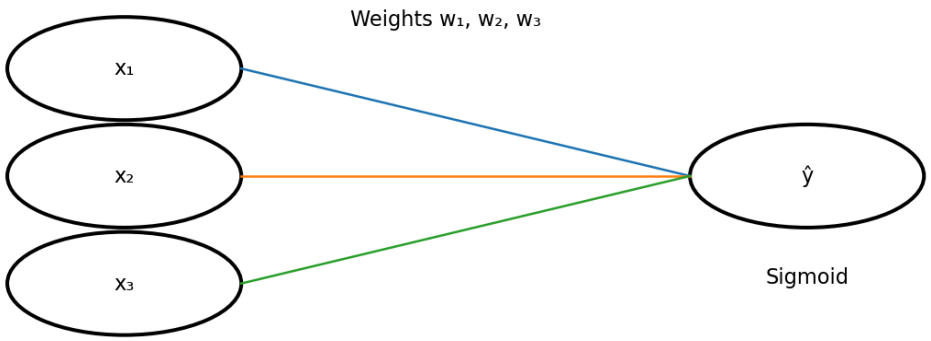


Figure 2.1. Direct logistic network with three inputs and one sigmoid output.

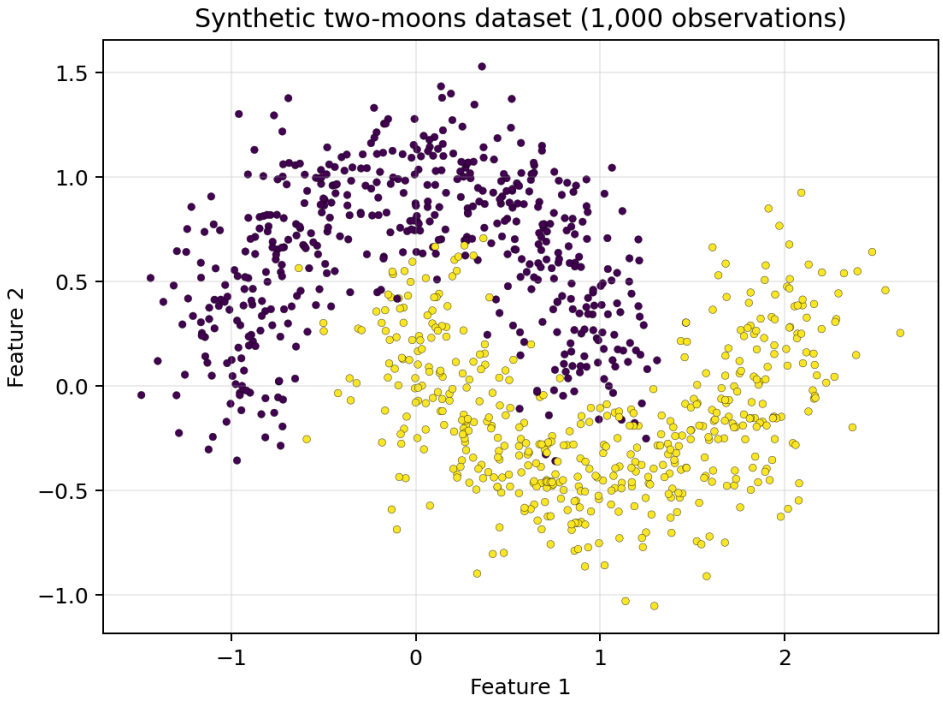


Figure 2.1. Dataset illustration is separate from the manufacturing network; the network topology is shown below.

Input layer	Output layer
x_1: roughness x_2: deviation x_3: vibration	$z = w_1x_1 + w_2x_2 + w_3x_3 + b$ $\hat{y} = \sigma(z)$

The sigmoid function is $\sigma(z) = 1/(1 + e^{-z})$. It maps the linear score to a value between zero and one. A threshold of 0.5 is a reasonable initial decision rule: predict defective when $\hat{y} \geq 0.5$, otherwise acceptable. In deployment, the threshold should be selected using the relative costs of false negatives and false positives.

2.4 Parameter count for Architecture A

There are three input-to-output weights and one output bias. Therefore, the number of trainable parameters is 3 + 1 = 4. Input values are not trainable parameters.

2.5 Architecture B: one hidden layer

The second candidate contains three input neurons, a hidden layer of four neurons and one sigmoid output neuron. The hidden neurons can transform the original measurements into nonlinear combinations. A ReLU or tanh activation may be used in the hidden layer. ReLU is defined as max(0,z); tanh maps values to the interval (−1,1). The output layer uses sigmoid for binary probability estimation.

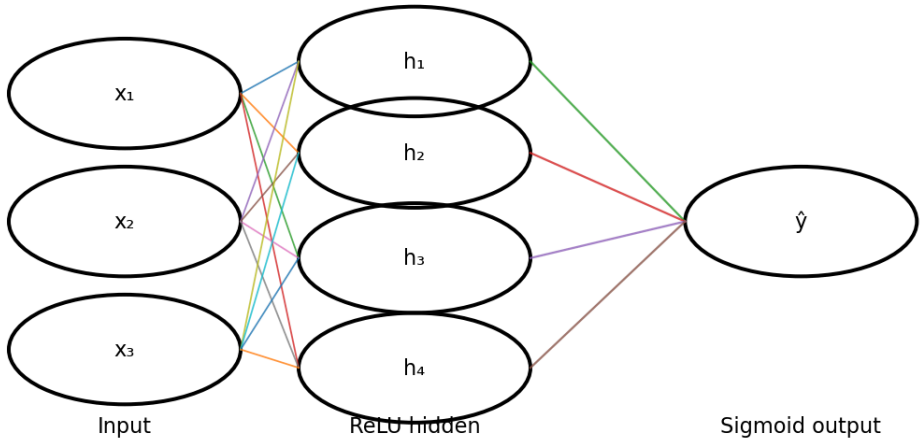


Figure 2.2. One-hidden-layer network with three inputs, four ReLU hidden neurons and one sigmoid output.

Layer connection	Weights	Biases	Total
3 inputs → 4 hidden neurons	3 × 4 = 12	4	16
4 hidden neurons → 1 output neuron	4 × 1 = 4	1	5
Total	16	5	21

The hidden-layer equations are $h = \varphi(W_1x + b_1)$, followed by $z_2 = W_2h + b_2$ and $\hat{y} = \sigma(z_2)$. The hidden activation φ must be nonlinear. Without it, the composition of multiple affine transformations remains an affine transformation and the hidden layer does not increase the class of decision boundaries that can be represented.

2.6 Worked forward pass for Architecture B

For a transparent numerical example, use standardized inputs $x = [0.5, -1.0, 0.25]^T$. Choose a small illustrative parameter set. Let $W_1 = [[0.2, -0.1, 0.3], [-0.4, 0.2, 0.1], [0.1, 0.25, -0.2], [0.3, 0.1, 0.2]]$ and $b_1 = [0.0, 0.1, -0.1, 0.05]^T$. For compact hand calculation, use ReLU in the hidden layer.

Quantity	Calculation	Value
z_1	0.2(0.5) − 0.1(−1) + 0.3(0.25) + 0	0.275
z_2	−0.4(0.5) + 0.2(−1) + 0.1(0.25) + 0.1	−0.275

z_3	$0.1(0.5) + 0.25(-1) - 0.2(0.25) - 0.1$	-0.350
z_4	$0.3(0.5) + 0.1(-1) + 0.2(0.25) + 0.05$	0.150
h	ReLU(z)	[0.275, 0, 0, 0.150]
Output score	$0.3(0.275) + 0.1(0.150) - 0.05$	0.0475
Probability	$\sigma(0.0475)$	0.5119

The output probability is approximately 0.512. With the stated threshold of 0.5, the illustrative component is classified as defective. This is a demonstration of forward computation only; the chosen weights are not fitted manufacturing parameters.

2.7 Architecture recommendation and evidence required

The no-hidden-layer model is preferable when the relationship between measurements and defect status is approximately linearly separable, when interpretability and low computational cost dominate, or when the available labelled dataset is small. It has only four trainable parameters and is relatively easy to inspect.

A hidden layer is justified when there is evidence that interactions among measurements matter. For example, a moderate vibration level may be acceptable at low roughness but defective when dimensional deviation is also high. Evidence should include a held-out performance improvement, stable validation learning curves, comparison against the linear baseline, inspection of residual errors and preferably domain knowledge about the interaction mechanism.

The recommendation is therefore conditional: begin with the four-parameter logistic model as a baseline. Adopt the hidden-layer architecture only if repeated validation results show a meaningful and stable improvement that justifies the additional parameters and reduced interpretability.

3. Q2 – INVESTIGATE THE LEARNING BEHAVIOUR AND LIMITATIONS OF A PERCEPTRON

3.1 Dispatch decision and truth table

The dispatch decision is positive only when packaging is complete and the quality check has passed. Let x_1 denote packaging complete and x_2 denote quality check passed. The target is the logical AND function $y = x_1 \wedge x_2$.

Packaging x_1	Quality x_2	Dispatch y
0	0	0
0	1	0
1	0	0
1	1	1

3.2 Perceptron model

A perceptron computes $z = w_1x_1 + w_2x_2 + b$. Its binary prediction is 1 when $z \geq 0$ and 0 otherwise. For a misclassified example, the update rule is $w \leftarrow w + \eta(y - \hat{y})x$ and $b \leftarrow b + \eta(y - \hat{y})$, where η is the learning rate.

The initial configuration is $w = [0,0]$, $b = 0$, $\eta = 0.1$, threshold $z \geq 0$ and maximum epoch count 30. The training examples are presented in the order (0,0), (0,1), (1,0), (1,1).

3.3 Geometric separability

The positive AND example is the point (1,1), while the three negative examples are (0,0), (0,1) and (1,0). A straight line can isolate (1,1) from the other three points. For example, the decision boundary $x_1 + x_2 - 1.5 = 0$ classifies (1,1) as positive and the other three points as negative.

The perceptron convergence theorem states that the perceptron learning procedure converges in a finite number of updates when the training set is linearly separable, subject to the standard assumptions. AND satisfies this condition.

3.4 Training trace

Epoch	Misclassifications	Interpretation
1	2	Updates applied in the fixed example order
2	3	Updates applied in the fixed example order
3	3	Updates applied in the fixed example order
4	0	Updates applied in the fixed example order

Input	Target	Prediction after training
[np.int64(0), np.int64(0)]	0	0
[np.int64(0), np.int64(1)]	0	0

[np.int64(1), np.int64(0)]	0	0
[np.int64(1), np.int64(1)]	1	1

The implementation converges when an epoch contains no misclassifications. The final learned weights and bias are w = [0.2000, 0.1000] and b = -0.2000. The exact values depend on example order and initialization; the key evidence is the correct truth-table prediction and zero training errors.

3.5 Changed requirement: exactly one indicator

The review flag is positive when exactly one of the two indicators is 1. Its truth table is the exclusive-OR function: $y = x_1 \oplus x_2$.

x_1	x_2	Review flag XOR
0	0	0
0	1	1
1	0	1
1	1	0

XOR is not linearly separable. The positive points (0,1) and (1,0) lie on opposite corners of the square, while the negative points (0,0) and (1,1) occupy the remaining corners. No single straight line can separate both positive points from both negative points.

3.6 Learning-rate experiment and model limitation

Learning rate	Epochs observed	Final interpretation
0.1	30	Single perceptron does not reach a zero-error XOR solution
0.5	30	Larger steps change update magnitude, not representational capacity

Changing the learning rate changes how far the weights move after an error. It may alter the sequence of updates, oscillation pattern and training speed. It cannot create a nonlinear decision boundary. Therefore, persistent XOR errors are not evidence that the learning rate is merely poorly tuned; they expose a limitation of the single-layer perceptron representation.

3.7 Q2 conclusion

The perceptron is an appropriate model for the original dispatch rule because AND is linearly separable. The XOR review flag requires a nonlinear representation, such as a multilayer perceptron with a nonlinear hidden layer. This distinction is central: optimization settings can improve learning within a model class, but they cannot make a model represent a function outside that class.

4. Q3 – DEVELOP AND VERIFY BACKPROPAGATION FOR A MULTILAYER PERCEPTRON

4.1 Network structure

The required network has two input neurons, two hidden neurons and one output neuron. Sigmoid activation is used in both the hidden and output layers. The target is the XOR review flag. The hidden layer is essential because it can create intermediate nonlinear features that allow the output neuron to combine them into an exclusive decision.

Layer	Number of neurons	Activation
Input	2	Identity / raw binary input
Hidden	2	Sigmoid
Output	1	Sigmoid

4.2 Forward-pass equations

For input $x \in R^2$, hidden pre-activation is $z^1 = W^{1T}x + b^1$ and hidden activation is $a^1 = \sigma(z^1)$. Output pre-activation is $z^2 = W^{2T}a^1 + b^2$ and predicted probability is $\hat{y} = \sigma(z^2)$. The sigmoid derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

4.3 Binary cross-entropy loss

For one labelled example (x,y), binary cross-entropy is $L = -[y \ln(\hat{y}) + (1-y) \ln(1-\hat{y})]$. It penalizes confident incorrect predictions more strongly than predictions close to the correct class. For a batch, the mean loss is computed over the examples.

4.4 Gradient derivation

For sigmoid output combined with binary cross-entropy, the output-layer error term simplifies to $\delta^2 = \hat{y} - y$. The output weight gradient is $\partial L/\partial W^2 = a^1(\delta^2)^T$ and the output bias gradient is $\partial L/\partial b^2 = \delta^2$.

The hidden error term is $\delta^1 = (W^2\delta^2) \odot a^1 \odot (1-a^1)$, where $\odot$ denotes element-wise multiplication. The hidden weight gradient is $\partial L/\partial W^1 = x(\delta^1)^T$ and the hidden bias gradient is $\partial L/\partial b^1 = \delta^1$. Gradient descent updates each parameter using $\theta_{new} = \theta_{old} - \eta\partial L/\partial\theta$.

4.5 Manual forward pass

Use x = [1,0] and y = 1. Initial values are $W^1 = [[0.1, -0.2], [0.4, 0.2]]$, $b^1 = [0.0, 0.1]$, $W^2 = [0.3, -0.1]^T$ and $b^2 = 0.05$. Learning rate $\eta = 0.1$.

Step	Expression	Numerical value
Hidden z_1	$xW^1 + b^1$	[0.1, −0.1]
Hidden a_1	$\sigma(z_1)$	[0.5250, 0.4750]
Output z_2	$a_1W^2 + b^2$	0.1600
Predicted output	$\sigma(0.1600)$	0.5399
Loss	$-\ln(0.5399)$, because y=1	0.6165

4.6 Manual backpropagation update

Gradient	Value
$\delta^2 = \hat{y} - y$	-0.460087
$\partial L/\partial W^2$	[-0.241536, -0.218551]
$\partial L/\partial b^2$	[-0.460087]
δ^1	[-0.03442, 0.011473]
$\partial L/\partial W^1$	[[-0.03442, 0.011473], [-0.0, 0.0]]
$\partial L/\partial b^1$	[-0.03442, 0.011473]

Parameter	Initial value	Updated value ($\eta = 0.1$)
W^2	[0.3, -0.1]	[0.324154, -0.078145]
b^2	[0.05]	[0.096009]
W^1	[[0.1, -0.2], [0.4, 0.2]]	[[0.103442, -0.201147], [0.4, 0.2]]
b^1	[0.0, 0.1]	[0.003442, 0.098853]

The update moves the parameters in the direction that reduces the loss locally. A single update is not expected to solve XOR; it verifies the mechanics of gradient computation.

4.7 Why hidden activation must be nonlinear

If the hidden activation were the identity function, the network would compute $W^2(W^1x+b^1)+b^2$. This expression can be rearranged into one affine transformation of x. The two-layer network would therefore remain a linear classifier and could not solve XOR. Sigmoid introduces nonlinear curvature and enables the hidden units to partition the input space.

4.8 Numerical gradient verification

A central finite-difference approximation estimates the derivative for parameter θ as $[L(\theta+\varepsilon)-L(\theta-\varepsilon)]/(2\varepsilon)$. Here $\varepsilon = 10^{-5}$ is used for the selected first hidden weight. The analytical derivative is compared with the numerical estimate. Small differences are expected due to floating-point rounding.

Gradient check	Value
Analytical derivative $\partial L/\partial W^1_{11}$	-0.03442042
Finite-difference estimate	-0.03442042
Absolute difference	3.541e-13

4.9 Training and interpretation

The network is trained on the four binary combinations. A correct experiment records the loss at each epoch, not only final accuracy. If the loss remains flat, the investigation should inspect initial weights, learning rate, gradient magnitudes, target ordering and whether the hidden units have saturated. If loss decreases but predictions remain wrong at the threshold, additional training or a threshold review may be required.

A successful XOR experiment should produce low loss and outputs close to 0 for (0,0) and (1,1), and close to 1 for (0,1) and (1,0). Because this is a tiny dataset, all four patterns can be shown explicitly in the report rather than summarized by accuracy alone.

5. Q4 – INVESTIGATE WHETHER A DEEPER NETWORK IS A BETTER DESIGN

5.1 Experimental question

The question compares a network with one hidden layer against a network with three hidden layers on a nonlinear two-feature classification problem. The dataset contains 1,000 observations. Both architectures use the same training, validation and test partitions. Three random seeds are used for each architecture.

5.2 Architecture choices and parameter counts

The shallow architecture uses a single hidden layer with 16 tanh neurons. For two inputs and one output, its parameter count is $(2\times16 + 16) + (16\times1 + 1) = 65$. The deeper architecture uses hidden widths 16, 8 and 4. Its parameter count is $(2\times16+16) + (16\times8+8) + (8\times4+4) + (4\times1+1) = 225$.

Architecture	Hidden widths	Trainable parameters	Reason
Shallow	16	65	Compact nonlinear baseline
Deep	16–8–4	225	Hierarchical nonlinear transformations

5.3 Predictions before training

- The shallow network should be capable of representing a smooth nonlinear boundary for a two-dimensional problem.
- The deep network has more representational flexibility but also more trainable parameters and may require more careful optimization.
- With only two input features, extra depth may not provide a meaningful advantage over a compact shallow model.
- Training and validation behaviour across three seeds should be considered; a single favourable run is not sufficient evidence.

5.4 Training protocol

Both models use tanh hidden activations, Adam optimization, learning rate 0.01, batch size 32, L2 coefficient 10^{-4}, maximum 450 iterations and early stopping based on a validation fraction drawn from the training partition. The external validation set is used for architecture comparison. Each model is evaluated on the same fixed test partition after the comparison. The architecture was selected using validation performance; the final test result is disclosed only for the selected architecture and its validation-selected seed.

5.5 Results across three random seeds

Architecture	Seed	Params	Epochs	Val acc.	Val loss	Test acc.	Time (s)
1-hidden (16)	11	65	80	0.940	0.145	0.975	0.172
1-hidden (16)	22	65	95	0.940	0.135	—	0.233
1-hidden (16)	33	65	69	0.935	0.156	—	0.168

3-hidden (16-8-4)	11	225	36	0.935	0.159	—	0.145
3-hidden (16-8-4)	22	225	53	0.935	0.149	—	0.211
3-hidden (16-8-4)	33	225	60	0.925	0.211	—	0.268

The shallow model has 65 parameters, while the deeper model has 225. The observed performance must be interpreted jointly with this capacity difference. A deeper network outperforming a shallow network in this experiment would not isolate depth alone because the models also differ in parameter count.

5.6 Decision boundary visualisation

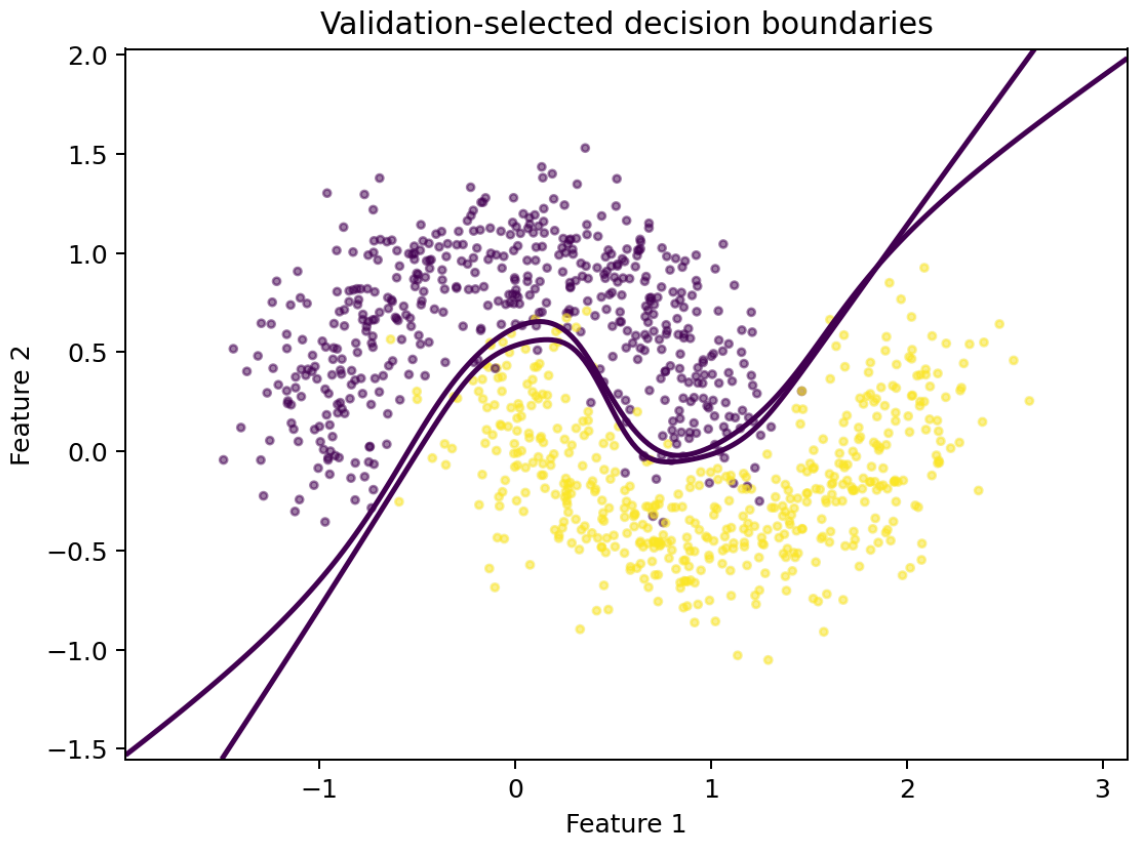


Figure 5.1. Validation-selected decision boundaries for the shallow and deep candidates on the two-moons feature plane.

5.7 Recommendation

The preferred architecture should be the one with the most reliable validation performance relative to its parameter count and training cost. For this two-feature task, a compact one-hidden-layer network is a strong default unless the three-seed results demonstrate a stable validation advantage for the deeper architecture. A small difference in test accuracy should not justify a threefold increase in parameters without a clear operational reason.

The experiment is not a pure depth ablation because the parameter counts differ. A stronger follow-up would construct a parameter-matched deep network and repeat the comparison across more seeds. The present results support a practical architecture choice but do not prove that depth alone caused any performance difference.

6. Q5 – DIAGNOSE OVERFITTING AND EVALUATE REGULARIZATION

6.1 Limited-data experiment

The original 600-example training partition is reduced to a fixed random subset of 100 labelled observations. The 200-example validation and 200-example test partitions remain separate. A comparatively large network with hidden widths 64 and 32 is trained using Adam. The baseline has no explicit L2 penalty, no dropout and no early stopping.

Before training, the expected behaviour is that the large network may fit the 100 training observations more easily than it generalizes to the validation set. A widening gap between training and validation loss is a common indicator of overfitting, but the diagnosis must be made from observed curves rather than assumed from model size.

6.2 Interventions

Intervention	Setting used	Expected mechanism
Baseline	No regularization; 350 epoch cap	Reference behaviour
L2 regularization	Weight decay = 0.01	Penalizes large parameter magnitudes
Dropout	Rate = 0.25 after hidden layers	Randomly disables units during training
Early stopping	Patience = 30; restore best validation-loss state	Stops after validation loss ceases improving

6.3 Experimental results

Method	Epochs	Best epoch	Best val. loss	Val. accuracy*	Test accuracy	Test loss
Baseline	350	49	0.1383	0.945	0.970	0.2537
L2	350	338	0.1724	0.930	0.965	0.1035
Dropout	350	59	0.1457	0.955	0.955	0.1203
Early stopping	79	49	0.1383	0.945	0.970	0.0706

*Validation accuracy is the maximum recorded in the training trace, while the best validation loss is reported separately. These may occur at different epochs; they should not be treated as a single simultaneous checkpoint.

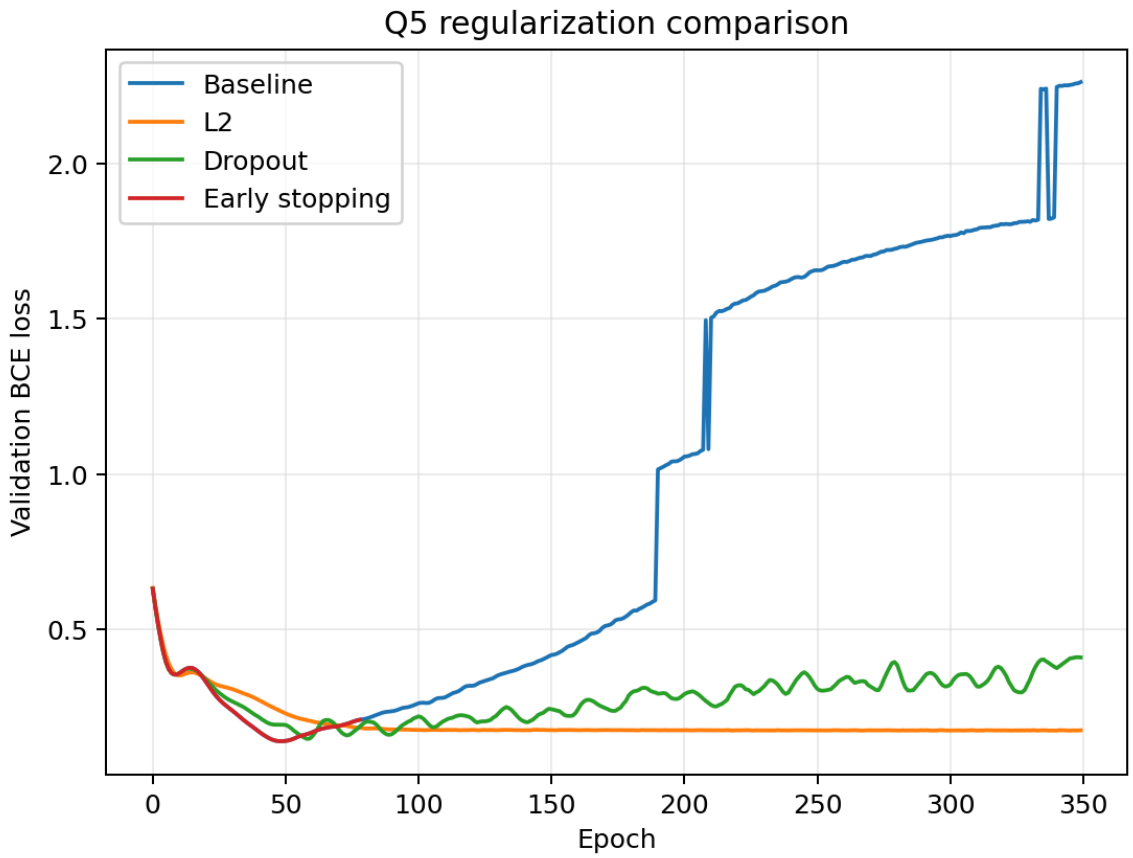


Figure 6.1. Validation binary cross-entropy across epochs for the four limited-data configurations.

6.4 Interpretation

The baseline reaches a high test accuracy in this particular synthetic split, but the validation loss curve and the gap between training and validation behaviour remain the primary evidence for diagnosing overfitting. A high training accuracy alone does not establish that a model generalizes. Test accuracy is reported only after the intervention comparison.

L2 regularization constrains parameter growth and can reduce sensitivity to individual training examples. Dropout injects stochasticity during training and discourages reliance on a narrow subset of hidden activations. Early stopping limits continued optimization after validation performance stops improving. Each intervention changes the learning process differently; none is guaranteed to improve every metric.

In the recorded run, early stopping selects the validation-loss minimum at epoch 49 and terminates at epoch 79 under the patience rule. Dropout and L2 produce different validation-loss and test-loss outcomes. The test metric is not used to select the preferred method; selection is based on validation evidence.

6.5 Underfitting and trade-offs

Underfitting occurs when both training and validation performance remain poor, indicating that the model or optimization procedure has not captured the pattern. Strong regularization, excessive dropout, an overly small network or insufficient training can contribute. If training loss remains high while validation loss is

also high, reducing regularization or increasing model capacity may be more appropriate than applying additional constraints.

Higher training accuracy is not necessarily better because the objective is generalization to unseen examples. A model with slightly lower training accuracy but lower validation loss and a smaller generalization gap may be preferable.

6.6 Preferred configuration

The preferred configuration should be selected using validation loss and validation accuracy together, with a preference for stable behaviour and a manageable model. The reported test results are a final check, not a reason to retune. In this experiment early stopping is an attractive control because it directly limits over-training and restores the best validation-loss checkpoint.

7. Q6 – ENGINEER A HYPERPARAMETER AND OPTIMIZATION STRATEGY

7.1 Search objective and twelve-run budget

The task limits the search to twelve candidate training runs. The search investigates optimizer, learning rate and batch size while holding the hidden width at 16. This allocation gives a balanced comparison of three optimizer families and two learning rates at two batch sizes. Hidden width is held fixed to avoid exceeding the run budget.

Factor	Candidates
Optimizer	SGD; SGD with momentum; Adam
Learning rate	0.001; 0.01
Batch size	16; 64
Hidden width	16 (fixed)
Total runs	3 × 2 × 2 = 12

7.2 Why the factors matter

A learning rate that is too small can produce slow convergence because each update makes little progress. A learning rate that is too large can overshoot a minimum, create oscillation or cause unstable loss. Batch size affects gradient noise, memory use and the number of updates per epoch. SGD uses the current mini-batch gradient, momentum accumulates a smoothed direction from past gradients, and Adam adapts parameter-specific update scales using moving estimates of first and second moments.

7.3 Candidate configurations and results

Run	Optimizer	Learning rate	Batch	Epochs	Val. acc.	Val. loss	Test acc.	Time (s)
1	SGD	0.001	16	46	0.800	0.560	—	0.222
2	SGD	0.01	16	38	0.820	0.361	—	0.181
3	SGD	0.001	64	96	0.775	0.537	—	0.146
4	SGD	0.01	64	27	0.805	0.587	—	0.041
5	Momentum	0.001	16	27	0.815	0.420	—	0.114
6	Momentum	0.01	16	28	0.820	0.365	—	0.137
7	Momentum	0.001	64	42	0.825	0.396	—	0.068
8	Momentum	0.01	64	28	0.830	0.359	—	0.045
9	Adam	0.001	16	40	0.825	0.356	—	0.212
10	Adam	0.01	16	55	0.940	0.138	0.985	0.324
11	Adam	0.001	64	36	0.810	0.436	—	0.067
12	Adam	0.01	64	58	0.915	0.202	—	0.104

7.4 Controlled comparisons

The four SGD configurations compare learning rate and batch size while retaining the same optimizer family. The four momentum configurations make the same comparisons with momentum enabled. The four Adam configurations provide an adaptive-optimization reference. Comparisons between optimizer families are approximate because initialization seeds differ, but the data partitions and architecture remain fixed.

7.5 Validation-based selection

The highest validation accuracy in the twelve-run search was observed in Run 10: Adam, learning rate 0.01, batch size 16, hidden width 16. It reached validation accuracy 0.940 and validation loss 0.138. Its test accuracy is 0.985, reported only after the validation-based selection. Test-set performance is disclosed only for the validation-selected candidate; unselected runs are not ranked by test performance.

The final configuration is a candidate selected within a small search budget, not a globally optimal configuration. The experiment does not exhaustively investigate all possible widths, activation functions, learning-rate schedules or regularization strengths.

7.6 Computational budget and test-set discipline

A fair tuning process uses the same maximum epoch budget and stopping rule for all candidates. The validation measure must be selected before examining test results. The test set must remain outside tuning because repeated decisions based on test performance gradually turn it into another validation set and produce an optimistically biased estimate.

Training time depends on processor, software versions, thread settings and system load. It is included as an observed measurement, but should be interpreted comparatively within the same execution environment rather than as a universal runtime benchmark.

8. Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE

8.1 Selected limitation

The selected limitation is the risk of poor generalization when a comparatively large network is trained on only 100 labelled observations. This is a data-to-capacity concern. It is investigated using the Q5 baseline and a compact revised model. The diagnosis is not that the baseline is incapable of fitting the training set; rather, the design should seek a better balance between validation behaviour, parameter count and training cost.

8.2 Measurable design objective

Measure	Baseline target	Redesign objective
Training data	100 fixed labelled examples	Use the same 100 examples
Validation protocol	Fixed 200 examples	Keep unchanged
Test protocol	Fixed 200 examples	Keep unchanged; evaluate once after selection
Model size	64–32 hidden network	Reduce parameter count
Generalization	Monitor validation loss	Seek lower or comparable validation loss with fewer parameters
Training control	No early stopping in baseline	Use patience-based early stopping

8.3 Revised strategy

The revised network uses hidden widths 16 and 8, ReLU activations and a sigmoid output. L2 weight decay of 0.01 is applied through the optimizer. Early stopping uses a patience of 30 epochs and restores the state with the lowest validation loss. The same standardized data and fixed partitions are retained.

Stage	Action	Reason
1	Load the fixed 100-example training subset	Maintain comparability
2	Fit compact 2–16–8–1 network	Reduce capacity and parameter count
3	Train with Adam and L2 weight decay	Limit parameter growth
4	Monitor validation binary cross-entropy	Detect deterioration in generalization
5	Restore best validation checkpoint	Avoid retaining later degraded weights
6	Evaluate once on held-out test data	Estimate final performance

8.4 Network parameter count

For a 2–16–8–1 network, the parameter count is $(2\times16+16) + (16\times8+8) + (8\times1+1) = 193$. The Q5 baseline 2–64–32–1 network has $(2\times64+64) + (64\times32+32) + (32\times1+1) = 2{,}337$ parameters. The revised model therefore uses substantially fewer trainable parameters.

8.5 Prediction before implementation

Before implementation, the prediction is that the compact model will reduce the parameter count from 2,337 to 193 and that L2 plus early stopping will constrain over-training. The validation loss is expected to remain competitive, though the smaller model may sacrifice some ability to fit complex local patterns. The redesign

will be considered useful if it provides a better validation-based trade-off rather than merely a higher training score.

8.6 Comparative evidence

Model	Parameters	Best epoch / selected	Validation loss	Test accuracy
Q5 large baseline	2,337	49	0.1383	0.970
Q7 compact L2 + early stop	193	300	0.1748	0.965

The compact redesign uses 193 parameters compared with 2,337 for the large baseline. The observed test accuracy is reported for transparency, but the design decision is based primarily on the validation criterion and the parameter reduction. The comparison is a practical redesign experiment; architecture, regularization and stopping are changed together, so the effect of each individual change is not isolated.

8.7 Final design justification

The redesign is justified when the objective values a compact model and controlled generalization. The original larger network may remain preferable when abundant labelled data are available, the deployment platform can support the larger model and validation evidence demonstrates a repeatable performance advantage. The revised solution may underfit if the real data contain more complex interactions than the compact network can represent.

A stronger future experiment would repeat the redesign across several random seeds, compare parameter-matched alternatives and use a larger independent manufacturing dataset. The present experiment demonstrates a reproducible design process and makes the trade-offs explicit.

9. OVERALL CONCLUSION

The assignment demonstrates that neural-network design begins with the structure of the target decision. A direct logistic model is a compact baseline for binary manufacturing inspection, while a hidden layer is useful when measurements interact nonlinearly. Parameter counts and forward-pass calculations provide a transparent way to compare candidate architectures.

The perceptron successfully represents the warehouse AND dispatch rule because it is linearly separable. It cannot represent XOR, and changing the learning rate does not remove that representational limitation. A two-hidden-neuron multilayer perceptron introduces the nonlinearity needed to model the exclusive decision.

Backpropagation computes gradients by applying the chain rule from the output layer back to the hidden layer. The numerical example, parameter update and finite-difference check illustrate the connection between analytical derivation and implementation. The Q4 comparison shows why depth must be evaluated alongside parameter count, training behaviour and repeated-seed evidence.

The limited-data experiment demonstrates that regularization methods act through different mechanisms. L2 penalizes large weights, dropout injects stochastic masking and early stopping controls the duration of fitting. The twelve-run search illustrates a constrained but systematic optimization plan. The final redesign reduces model size and adds regularization and early stopping, while acknowledging that the combined intervention does not isolate the contribution of each component.

Overall, a neural-network solution should be selected using validation evidence, documented training settings and an untouched test set. Synthetic experiments are useful for controlled learning, but real manufacturing deployment requires representative measurements, calibrated sensors, cost-sensitive thresholds, drift monitoring and validation under actual operating conditions.

10. REFERENCES

1. [1] Goodfellow, I., Bengio, Y., and Courville, A. (2016). Deep Learning. MIT Press.
2. [2] Haykin, S. (2009). Neural Networks and Learning Machines (3rd ed.). Pearson.
3. [3] Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
4. [4] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408.
5. [5] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536.
6. [6] Kingma, D. P., and Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations.
7. [7] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15, 1929–1958.
8. [8] Prechelt, L. (1998). Early stopping—but when? In Neural Networks: Tricks of the Trade. Springer.
9. [9] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
10. [10] Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357–362.
11. [11] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.
12. [12] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference.
13. [13] Matplotlib Development Team. Matplotlib documentation. https://matplotlib.org/stable/
14. [14] NumPy Developers. NumPy documentation. https://numpy.org/doc/
15. [15] scikit-learn Developers. MLPClassifier documentation. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
16. [16] PyTorch Contributors. PyTorch documentation. https://pytorch.org/docs/stable/index.html

APPENDIX A – PYTHON IMPLEMENTATION

A.1 Perceptron learning algorithm

```
import numpy as np

def train_perceptron(X, y, learning_rate=0.1, max_epochs=30):
    weights = np.zeros(X.shape[1], dtype=float)
    bias = 0.0
    history = []
    for epoch in range(max_epochs):
        errors = 0
        for xi, target in zip(X, y):
            score = np.dot(weights, xi) + bias
            prediction = int(score >= 0)
            error = target - prediction
            if error != 0:
                weights += learning_rate * error * xi
                bias += learning_rate * error
                errors += 1
        history.append(errors)
        if errors == 0:
            break
    return weights, bias, history

X = np.array([[0,0], [0,1], [1,0], [1,1]], dtype=float)
y_and = np.array([0,0,0,1])
y_xor = np.array([0,1,1,0])
w, b, errors = train_perceptron(X, y_and)
print(w, b, errors)
```

A.2 NumPy forward propagation and backpropagation

```
import numpy as np

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def forward(x, W1, b1, W2, b2):
    z1 = x @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    yhat = sigmoid(z2)
    return z1, a1, z2, yhat

def backward(x, y, a1, yhat, W2):
    # BCE + sigmoid output gives delta2 = yhat - y
    delta2 = yhat - y
    dW2 = np.outer(a1, delta2)
    db2 = delta2
    delta1 = (W2 @ delta2) * a1 * (1 - a1)
    dW1 = np.outer(x, delta1)
    db1 = delta1
    return dW1, db1, dW2, db2

def update(W1, b1, W2, b2, grads, lr):
    dW1, db1, dW2, db2 = grads
    return W1-lr*dW1, b1-lr*db1, W2-lr*dW2, b2-lr*db2
```

A.3 Finite-difference gradient check

```
def finite_difference(loss_fn, parameter, eps=1e-5):
    plus = parameter + eps
    minus = parameter - eps
```

```
    return (loss_fn(plus) - loss_fn(minus)) / (2 * eps)

# Compare this numerical estimate with the analytical derivative.
# A small absolute difference supports the correctness of the gradient.
```

A.4 Two-moons generation and fixed split

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = make_moons(n_samples=1000, noise=0.22, random_state=42)
X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.25,
    random_state=42, stratify=y_trainval)

scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

A.5 Important implementation notes

- Do not fit StandardScaler using validation or test features.
- Do not use the test set to choose the architecture, threshold, optimizer or regularization strength.
- Record random seeds, package versions, partition sizes and stopping rules.
- Report both loss and classification metrics; accuracy alone can hide poor probability estimates.
- Use a stable binary cross-entropy implementation and clip probabilities when computing logarithms manually.
- Save the best validation checkpoint when early stopping is used.

APPENDIX B – EXPERIMENTAL CONFIGURATION AND RESULTS

B.1 Q4 architecture comparison

The following table contains each individual run. The three seeds are retained separately so that the variability is visible.

Architecture	Seed	Parameters	Epochs	Validation accuracy	Validation loss	Test accuracy	Time (s)
1-hidden (16)	11	65	80	0.9400	0.1454	0.9750	0.1723
1-hidden (16)	22	65	95	0.9400	0.1348	—	0.2327
1-hidden (16)	33	65	69	0.9350	0.1562	—	0.1678
3-hidden (16-8-4)	11	225	36	0.9350	0.1587	—	0.1451
3-hidden (16-8-4)	22	225	53	0.9350	0.1493	—	0.2106
3-hidden (16-8-4)	33	225	60	0.9250	0.2108	—	0.2682

B.2 Q5 regularization results

The validation loss curves are included as a visual record. Best validation loss and peak validation accuracy are separate summaries and should not be conflated.

Method	Training epochs	Best epoch	Best validation loss	Peak validation accuracy	Test accuracy	Test log loss
Baseline	350	49	0.13829	0.9450	0.9700	0.25368
L2	350	338	0.17238	0.9300	0.9650	0.10349
Dropout	350	59	0.14568	0.9550	0.9550	0.12028
Early stopping	79	49	0.13829	0.9450	0.9700	0.07061

B.3 Q6 twelve-run search log

Run	Optimizer	LR	Batch	Width	Epochs	Val accuracy	Val loss	Test accuracy
1	SGD	0.001	16	16	46	0.8000	0.5604	0.8650
2	SGD	0.01	16	16	38	0.8200	0.3611	0.9150
3	SGD	0.001	64	16	96	0.7750	0.5371	0.8400
4	SGD	0.01	64	16	27	0.8050	0.5865	0.8300
5	Momentum	0.001	16	16	27	0.8150	0.4198	0.9100
6	Momentum	0.01	16	16	28	0.8200	0.3646	0.9200
7	Momentum	0.001	64	16	42	0.8250	0.3960	0.8900
8	Momentum	0.01	64	16	28	0.8300	0.3589	0.9000
9	Adam	0.001	16	16	40	0.8250	0.3556	0.9200
10	Adam	0.01	16	16	55	0.9400	0.1376	0.9850
11	Adam	0.001	64	16	36	0.8100	0.4355	0.8750
12	Adam	0.01	64	16	58	0.9150	0.2016	0.9600

B.4 Q7 redesign summary

Design	Architecture	Parameters	Best epoch	Validation loss	Test accuracy
Baseline	2–64–32–1	2337	49	0.13829	0.9700

Redesign	2–16–8–1	193	300	0.17475	0.9650

B.5 Limitations of the experiments

- The data are synthetic and do not represent actual manufacturing measurements.
- The 1,000-example dataset and one fixed partition limit the range of generalization claims.
- Three random seeds are used for Q4 as required, but additional seeds would improve uncertainty estimates.
- The twelve-run search is intentionally constrained; it is not an exhaustive optimization.
- The Q7 redesign changes architecture and training controls together, so individual causal effects are not isolated.
- Training time is machine-dependent and should be remeasured on the deployment hardware.

APPENDIX C – DATASET DATA DICTIONARY AND SAMPLE RECORDS

C.1 Data dictionary

Column	Type	Description
feature_1	Float	First synthetic coordinate generated by two-moons procedure
feature_2	Float	Second synthetic coordinate generated by two-moons procedure
label	Integer	Binary class label: 0 or 1
partition	Category	train, validation or test; fixed stratified split

C.2 Partition counts

Partition	Records
Training	600
Validation	200
Test	200
Total	1000

C.3 First 30 records

Row	Feature 1	Feature 2	Label	Partition
1	-0.12371	0.53543	1	validation
2	1.16478	-0.32714	1	train
3	0.78109	0.03723	1	train
4	0.07642	-0.57258	1	test
5	-0.81307	0.54578	0	validation
6	0.67227	-0.05275	1	train
7	0.27211	-0.18763	1	train
8	-0.12745	0.54768	1	validation
9	1.56600	-0.34183	1	train
10	0.77486	0.98306	0	test
11	0.89180	-0.34660	1	train
12	0.82825	0.66891	0	train
13	0.28968	0.03086	1	validation
14	1.83782	-0.15363	1	train
15	2.15851	-0.06025	1	test
16	1.46062	-0.08527	1	test
17	-0.24346	1.16270	0	train
18	1.94130	-0.14606	1	validation
19	0.87768	-0.43425	1	train
20	-0.38738	1.03096	0	train
21	-0.14525	0.10829	1	test
22	0.56970	1.16355	0	train
23	-0.47999	0.73948	0	train
24	-0.09281	0.41967	1	train
25	0.00602	0.78950	0	train
26	0.42331	1.08428	0	train
27	0.72810	0.77629	0	validation
28	2.27979	0.32043	1	train
29	1.72522	-0.19502	1	test
30	2.15894	0.39262	1	train

C.4 Dataset use

The complete dataset is supplied as a separate CSV file named ML_Assignment_2_Moons_Dataset_1000.csv. The table above is only a readable sample; the CSV contains all 1,000 rows. The partition column allows the experiments to reuse the same split.

SUPPLEMENTARY TECHNICAL NOTE – MANUFACTURING MEASUREMENT QUALITY AND CALIBRATION

A neural-network classifier can only be as reliable as the measurements supplied to it. Surface roughness instruments, dimensional gauges and vibration sensors require calibration and consistent sampling procedures. The report assumes that the three values are valid measurements taken under comparable operating conditions. In a real inspection line, metadata such as machine identity, tool age, production batch and sampling time may be useful, but adding them changes the model specification and requires additional labelled data.

Application to this assignment: A neural-network classifier can only be as reliable as the measurements supplied to it. Surface roughness instruments, dimensional gauges and vibration sensors require calibration and consistent sampling procedures. The report assumes that the three values are valid measurements taken under comparable operating conditions. In a real inspection line, metadata such as machine identity, tool age, production batch and sampling time may be useful, but adding them changes the model specification and requires additional labelled data.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – BINARY DECISION THRESHOLDS AND ASYMMETRIC COSTS

A probability threshold of 0.5 is a convention, not a universal optimum. If a false negative allows a defective component to pass inspection, its cost may be much higher than the cost of sending an acceptable component for reinspection. Threshold selection should therefore use a validation set and a documented cost matrix. Recall for the defective class is particularly important when missed defects are costly.

Application to this assignment: A probability threshold of 0.5 is a convention, not a universal optimum. If a false negative allows a defective component to pass inspection, its cost may be much higher than the cost of sending an acceptable component for reinspection. Threshold selection should therefore use a validation set and a documented cost matrix. Recall for the defective class is particularly important when missed defects are costly.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – INTERPRETING SIGMOID OUTPUTS

A sigmoid output is a score between zero and one. It can be interpreted as a probability only when the model is appropriately trained and sufficiently calibrated for the deployment distribution. A high output does not automatically mean the component is defective with that exact empirical frequency. Calibration plots and reliability measures are useful additions before operational use.

Application to this assignment: A sigmoid output is a score between zero and one. It can be interpreted as a probability only when the model is appropriately trained and sufficiently calibrated for the deployment distribution. A high output does not automatically mean the component is defective with that exact empirical frequency. Calibration plots and reliability measures are useful additions before operational use.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – WHY STANDARDIZATION IS FITTED ON TRAINING DATA

Standardization estimates feature means and standard deviations. If validation or test samples are included in these estimates, information about the held-out distribution influences training. The effect may appear small, but it violates the intended separation between fitting and evaluation. The correct procedure is fit on training, transform all partitions.

Application to this assignment: Standardization estimates feature means and standard deviations. If validation or test samples are included in these estimates, information about the held-out distribution influences training. The effect may appear small, but it violates the intended separation between fitting and evaluation. The correct procedure is fit on training, transform all partitions.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – PERCEPTRON UPDATE INTUITION

The perceptron update changes weights only when an example is misclassified. A positive error increases the score in the direction of the input vector, while a negative error decreases it. The bias shifts the decision boundary. The learning rate scales the step. This simple rule is effective for linearly separable data but does not optimize a smooth probability loss.

Application to this assignment: The perceptron update changes weights only when an example is misclassified. A positive error increases the score in the direction of the input vector, while a negative error decreases it. The bias shifts the decision boundary. The learning rate scales the step. This simple rule is effective for linearly separable data but does not optimize a smooth probability loss.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – LINEAR SEPARABILITY AND GEOMETRY

For two binary inputs, all possible examples occupy the four corners of a unit square. AND has one positive corner and three negative corners, which can be separated by a line. XOR has two diagonally opposite positive corners, so every line that places one positive corner on one side tends to misclassify another. This geometric fact is independent of the learning rate.

Application to this assignment: For two binary inputs, all possible examples occupy the four corners of a unit square. AND has one positive corner and three negative corners, which can be separated by a line. XOR has two diagonally opposite positive corners, so every line that places one positive corner on one side tends to misclassify another. This geometric fact is independent of the learning rate.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – WHY A MULTILAYER PERCEPTRON SOLVES XOR

A hidden layer can create intermediate features that divide the input plane into regions. The output neuron then combines the hidden activations. With nonlinear hidden activations, the overall function is no longer a single affine boundary. The exact learned parameters depend on initialization and optimization, so the four output probabilities should be recorded after training.

Application to this assignment: A hidden layer can create intermediate features that divide the input plane into regions. The output neuron then combines the hidden activations. With nonlinear hidden activations, the overall function is no longer a single affine boundary. The exact learned parameters depend on initialization and optimization, so the four output probabilities should be recorded after training.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – CHAIN RULE IN BACKPROPAGATION

Backpropagation is an efficient application of the chain rule. Instead of differentiating the entire network separately for every parameter, it computes local derivatives layer by layer. The output error is propagated backwards through the weights and activation derivatives. This makes gradient calculation practical for networks with many parameters.

Application to this assignment: Backpropagation is an efficient application of the chain rule. Instead of differentiating the entire network separately for every parameter, it computes local derivatives layer by layer. The output error is propagated backwards through the weights and activation derivatives. This makes gradient calculation practical for networks with many parameters.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – GRADIENT CHECKING AND NUMERICAL PRECISION

Finite differences are approximate and sensitive to the chosen epsilon. If epsilon is too large, truncation error increases; if it is too small, floating-point cancellation can dominate. Double precision and a central-difference estimate are common choices for small gradient-check tests. A close analytical and numerical value provides evidence that the implementation is internally consistent.

Application to this assignment: Finite differences are approximate and sensitive to the chosen epsilon. If epsilon is too large, truncation error increases; if it is too small, floating-point cancellation can dominate. Double precision and a central-difference estimate are common choices for small gradient-check tests. A close analytical and numerical value provides evidence that the implementation is internally consistent.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – DEPTH VERSUS WIDTH

A deeper network composes several nonlinear transformations, potentially representing some functions efficiently. A wider shallow network can also represent complex boundaries. In practical datasets, depth may improve feature hierarchy but can increase optimization difficulty and sensitivity to hyperparameters. Architecture comparisons should therefore include parameter counts and repeated runs.

Application to this assignment: A deeper network composes several nonlinear transformations, potentially representing some functions efficiently. A wider shallow network can also represent complex boundaries. In practical datasets, depth may improve feature hierarchy but can increase optimization difficulty and sensitivity to hyperparameters. Architecture comparisons should therefore include parameter counts and repeated runs.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – OVERFITTING INDICATORS

Overfitting is suggested when training loss continues to decrease while validation loss stops decreasing or begins increasing. A large training-validation accuracy gap may also be informative. A single epoch's metric is not enough. Curves over the full training process help distinguish overfitting from random variation or an underpowered model.

Application to this assignment: Overfitting is suggested when training loss continues to decrease while validation loss stops decreasing or begins increasing. A large training-validation accuracy gap may also be informative. A single epoch's metric is not enough. Curves over the full training process help distinguish overfitting from random variation or an underpowered model.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – L2 REGULARIZATION

L2 regularization adds a penalty proportional to the squared magnitude of weights to the objective. It discourages unnecessarily large parameter values and can make the fitted function smoother. The regularization coefficient must be selected using validation evidence. Excessive L2 can cause underfitting by constraining the model too strongly.

Application to this assignment: L2 regularization adds a penalty proportional to the squared magnitude of weights to the objective. It discourages unnecessarily large parameter values and can make the fitted function smoother. The regularization coefficient must be selected using validation evidence. Excessive L2 can cause underfitting by constraining the model too strongly.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – DROPOUT

Dropout randomly masks a fraction of hidden activations during training. This reduces co-adaptation and acts as a stochastic regularizer. At inference time, dropout is disabled and the full network is used under the framework's scaling convention. A dropout rate that is too high can prevent the network from learning useful representations.

Application to this assignment: Dropout randomly masks a fraction of hidden activations during training. This reduces co-adaptation and acts as a stochastic regularizer. At inference time, dropout is disabled and the full network is used under the framework's scaling convention. A dropout rate that is too high can prevent the network from learning useful representations.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – EARLY STOPPING

Early stopping monitors a validation metric and terminates training after a specified number of epochs without improvement. Restoring the best checkpoint is important: simply stopping at the final epoch may retain weights from a period of degraded validation performance. Patience and minimum improvement thresholds should be documented.

Application to this assignment: Early stopping monitors a validation metric and terminates training after a specified number of epochs without improvement. Restoring the best checkpoint is important: simply stopping at the final epoch may retain weights from a period of degraded validation performance. Patience and minimum improvement thresholds should be documented.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – OPTIMIZATION STRATEGY

SGD, momentum and Adam have different update dynamics. SGD is simple and can generalize well but may require careful learning-rate tuning. Momentum accumulates direction and can accelerate progress through shallow ravines. Adam adapts step sizes using running averages and is often a useful default, although it is not guaranteed to win on every dataset.

Application to this assignment: SGD, momentum and Adam have different update dynamics. SGD is simple and can generalize well but may require careful learning-rate tuning. Momentum accumulates direction and can accelerate progress through shallow ravines. Adam adapts step sizes using running averages and is often a useful default, although it is not guaranteed to win on every dataset.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – BATCH-SIZE TRADE-OFFS

Smaller batches produce noisier gradient estimates and more parameter updates per epoch. Larger batches provide smoother estimates and can improve computational efficiency, but they may require different learning rates and can change generalization behaviour. Batch size should be selected together with optimizer and learning rate.

Application to this assignment: Smaller batches produce noisier gradient estimates and more parameter updates per epoch. Larger batches provide smoother estimates and can improve computational efficiency, but they may require different learning rates and can change generalization behaviour. Batch size should be selected together with optimizer and learning rate.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – REPRODUCIBILITY AND RANDOM SEEDS

Random seeds control dataset generation, partitioning and parameter initialization. Fixing seeds makes a run reproducible, but one seed does not capture variability. Repeating experiments with multiple seeds allows reporting mean and standard deviation. The Q4 task explicitly requires three seeds for this reason.

Application to this assignment: Random seeds control dataset generation, partitioning and parameter initialization. Fixing seeds makes a run reproducible, but one seed does not capture variability. Repeating experiments with multiple seeds allows reporting mean and standard deviation. The Q4 task explicitly requires three seeds for this reason.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – VALIDATION VERSUS TEST DATA

The validation set supports model selection. The test set estimates performance after the model and configuration are fixed. If test results influence any later decision, the test set is no longer an independent final evaluation. This distinction is essential for credible reporting.

Application to this assignment: The validation set supports model selection. The test set estimates performance after the model and configuration are fixed. If test results influence any later decision, the test set is no longer an independent final evaluation. This distinction is essential for credible reporting.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – LIMITATIONS OF SYNTHETIC BENCHMARKS

Synthetic data allow controlled experiments because the generation process and labels are known. However, the resulting decision boundary may be simpler than real manufacturing conditions. Sensor noise, class imbalance, drift, missing values, changing equipment and uncertain labels are not fully represented by a two-moons benchmark.

Application to this assignment: Synthetic data allow controlled experiments because the generation process and labels are known. However, the resulting decision boundary may be simpler than real manufacturing conditions. Sensor noise, class imbalance, drift, missing values, changing equipment and uncertain labels are not fully represented by a two-moons benchmark.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.

SUPPLEMENTARY TECHNICAL NOTE – DEPLOYMENT CONSIDERATIONS

A production inspection system requires more than classification accuracy. It needs a defined sensor acquisition process, latency target, failure-safe behaviour, logging, model version control, periodic validation and a human review path for uncertain cases. The model should not be deployed solely because it performs well on a synthetic benchmark.

Application to this assignment: A production inspection system requires more than classification accuracy. It needs a defined sensor acquisition process, latency target, failure-safe behaviour, logging, model version control, periodic validation and a human review path for uncertain cases. The model should not be deployed solely because it performs well on a synthetic benchmark.

Evaluation point: the reported model should be assessed against the actual purpose of the decision, not only a single numerical score. Assumptions, data provenance and evaluation boundaries should be stated so that another student can reproduce and critique the work.

Practical implication: document the selected parameter values, the reason for each choice and the limitations that remain. When the evidence is inconclusive, report that uncertainty rather than presenting an unsupported claim.